

Lab 7: Recursion

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Base cases, recursive traces, binary search, Fibonacci

Objectives

- Identify the base case(s) and recursive case(s) of a recursive method.
- Draw a recursion trace to understand how the call stack unwinds.
- Implement and analyse: factorial, binary search, power, and Fibonacci.
- Recognise the exponential cost of naive Fibonacci and apply memoisation.

Quick Reference

Anatomy of a recursive method.

1. **Base case** — returns without a recursive call.
2. **Recursive case** — calls itself with a *strictly smaller* sub-problem.

Factorial.

```
static long factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

Binary search (recursive).

```
static int binSearch(int[] a, int lo, int hi, int target) {  
    if (lo > hi) return -1;  
    int mid = (lo + hi) / 2;  
    if (a[mid] == target) return mid;  
    else if (target < a[mid]) return binSearch(a, lo, mid-1, target);  
    else return binSearch(a, mid+1, hi, target);  
}
```

Complexity. Factorial: $O(n)$. Binary search: $O(\log n)$. Naive Fibonacci: $O(2^n)$.

Quick Experiments (Try It)

Try It 1: Stack overflow — how deep can you go?

Call `factorial(n)` for increasing `n` (try 10 000, 100 000). At what value does a `StackOverflowError` occur?

Try It 2: Counting recursive calls

Add a static counter `int calls = 0` to your Fibonacci and increment at each call. Compare for `fib(10)`, `fib(20)`, `fib(30)`. How does the count grow?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Factorial and Power (20 min)

Implement:

- `long factorial(int n)` — recursive.
- `long power(long base, int exp)` — recursive; `power(b,0) = 1`.

Test: `factorial(10) = 3628800`, `power(2,10) = 1024`. Draw the recursion trace for `power(3,4)`.

In-Lab Exercise 2: Recursive Binary Search (20 min)

Implement `int binSearch(int[] a, int lo, int hi, int target)`. Test on a sorted array of 15 elements: search for the middle, first, last, and a missing element. Count recursive calls for each search.

In-Lab Exercise 3: Fibonacci: Naive vs. Memoised (40 min)

1. Implement naive `long fibNaive(int n)`.
2. Implement memoised `long fibMemo(int n, long[] memo)`: initialise `memo` to `-1`; store each result before returning.
3. Time both for $n = 10, 20, 30, 40$ using `System.nanoTime()`.

n	fibNaive (ns)	fibMemo (ns)	Speedup
10			
20			
30			
40			

In-Lab Exercise 4: Sum of Digits and Digit Count (25 min)

Write two recursive methods (base case: $n < 10$; recursive case uses $n / 10$):

- `int digitSum(int n)` — sum of all digits.
- `int countDigits(int n)` — number of digits.

Test: `digitSum(1234) = 10`, `countDigits(100) = 3`.

AI Collaboration

Suggested prompts:

- “Draw the recursion tree for `fib(5)`, counting duplicate sub-problems.”
- “Explain memoisation: how does it change Fibonacci from $O(2^n)$ to $O(n)$?”

Verify: Have the AI generate memoised Fibonacci. Run it alongside yours for $n = 40$. Do they agree? Is the AI’s version iterative or recursive?

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Recursion trace for power

Draw the full recursion trace (call stack) for `power(2, 5)`. Show each call with its arguments and the return value as the stack unwinds.

Trace & Predict 2: Binary search trace

Trace `binSearch(a, 0, 7, 19)` on $a = [3, 7, 11, 15, 19, 23, 27, 31]$. For each call show: `lo`, `hi`, `mid`, and the comparison result. How many calls are made?

Write Code on Paper 1: Recursive string reverse

Write `String reverse(String s)` recursively. Base case: `s` is empty or one character. Recursive case: last character + `reverse(s without last character)`.

Submission. Save files as `lab7_ex*.java` and submit on the course site.